Minimal Equality Saturation for FPGA Datapath Optimization

Zhenghao Yang

Queen's University, Kingston, Canada
24yl6@queensu.ca

Abstract — We present **hwine**[1], a declarative equality-saturation engine for FPGA datapath optimization that uses a single associativity rewrite rule—with no machine learning and no benchmark-specific rules—to automatically discover optimal hardware structures. The system operates at the register-transfer level (RTL), using e-graph equality saturation with an FPGA-aware cost model. We evaluate hwine against the UCSD FIR benchmark suite, comparing against 11 hand-optimized HLS implementations from Prof. Kastner's group. At all 10 matching sizes, hwine achieves identical Fmax (308 MHz) to Kastner's expert designs, using fewer LUTs on 6 of 10 sizes (e.g., 213 vs. 246 at 6 taps, 1299 vs. 2041 at 41 taps, 4757 vs. 6821 at 152 taps). On larger pure adder trees, hwine maintains 226 MHz Fmax from 20–152 taps—a 2.6× speedup over serial. Runtime scales efficiently to 512 inputs in 10 ms. Compared to ROVER (187 rules) and PrefixRL (32,000 GPU-hours), hwine demonstrates that a single algebraic rule with FPGA-aware extraction achieves expert-level results. All designs verified on Xilinx 7-series via Vivado 2026.1 with zero errors and zero warnings.

Index Terms — e-graph, equality saturation, FPGA, datapath optimization, RTL, hardware architecture, FIR filter, Vivado, HLS

1. Introduction

Field-programmable gate arrays (FPGAs) offer unique opportunities for custom datapath design, but achieving optimal quality of results (QoR) requires exploring a vast space of equivalent micro-architectures. Designers must manually decide on adder tree topologies, pipeline stage insertion points, DSP versus LUT resource allocation, and coefficient multiplication strategies—decisions that interact combinatorially and resist local optimization heuristics.

Existing approaches to automating this exploration fall into three categories. **Traditional logic synthesis** tools (Yosys, Vivado, Synopsys DC) apply destructive rewriting at the gate level and suffer from the **phase-ordering problem**: the order of rewrites determines the final result, and finding the optimal order is NP-hard. **Hand-crafted e-graph systems** such as ROVER [1] encode domain expertise as 187 manually-designed rewrite rules, achieving up to 63% area reduction on Intel benchmarks but requiring significant expert effort per operator family. **Reinforcement learning approaches** such as PrefixRL [2] use deep RL to search design spaces without domain knowledge, but at the cost of 32,000 GPU-hours of training and limited generalizability beyond prefix circuits.

We propose a fourth approach: **minimal declarative rewriting**—using a single algebraic rule (associativity), combined with FPGA-aware cost-driven extraction and automated structural pipelining, to recover hardware structures that normally require expert RTL or HLS design. Our system, hwine[2], makes three contributions:

1. **One rule suffices for associative datapaths.** A single associativity rewrite rule, applied via e-graph equality saturation, discovers balanced reduction trees, DSP cascade structures, and common subexpression sharing that normally require hand-coded optimization passes.
2. **FPGA-aware extraction bridges the gap to real hardware.** hwine uses a delay-weighted cost function calibrated against Vivado synthesis results, with automatic per-operation pipeline insertion at configurable granularity.

[1] **hwine** stands for hardware is not enough — and also hwine is not enough, a doubly self-referential name.
[2] Throughout this paper, we use the lowercase name **hwine** to refer to our tool, following the convention of systems like egg, seer, and yosys.

3. **Expert-level results on standard benchmarks across all scales.** hwine matches Kastner's Fmax (308 MHz) at all 10 tested sizes, using 13–38% fewer LUTs on 6 of 10 sizes. For pure adder trees, pipelined Fmax stays constant at 226 MHz across 20–152 taps.

We evaluate hwine across three experiment categories: pure adder tree scaling, DSP-based multiply-accumulate FIR designs, and constant-folded shift-add FIR designs versus the complete UCSD Kastner FIR benchmark suite.

2. Background

2.1. Equality Saturation

An e-graph [3] is a data structure that compactly represents many equivalent programs. It consists of **e-classes** (equivalence classes of expressions) containing **e-nodes** (operations whose children are e-class IDs). The key operation is **equality saturation**: repeatedly applying rewrite rules to add new e-nodes to existing e-classes, and **merging** e-classes when equivalence is discovered. After saturation, **extraction** selects the optimal expression from each e-class according to a cost function.

Unlike destructive rewriting (where each transformation replaces the original expression), equality saturation preserves all equivalent forms simultaneously, avoiding the phase-ordering problem entirely. The cost model determines which equivalent form is selected during extraction, decoupling the **exploration** of equivalent structures from the **selection** of the optimal one.

2.2. FPGA Arithmetic Primitives

FPGAs implement arithmetic through two primary resources. **Look-up tables** (LUTs) are 4-to-6-input programmable logic elements; modern Xilinx 7-series devices use 6-input LUTs. **Carry chains** (CARRY4) provide fast dedicated routing between adjacent LUTs for efficient addition. **DSP blocks** (DSP48E1 in 7-series) are hardened multiply-accumulate units with 25×18-bit multipliers, 48-bit accumulators, and cascaded interconnects (PCIN/PCOUT) that enable chaining multiple DSPs without routing through general interconnect.

2.3. Related Work

ROVER [1] is the most directly comparable system, using e-graph rewriting with 187 rules and ILP-based extraction for ASIC datapath optimization, achieving 63% area reduction on Intel benchmarks. **PrefixRL** [2] uses deep Q-learning to optimize parallel prefix circuits (25% lower area at same delay), deployed in NVIDIA's H100 GPU. **ASPEN** [4] uses LLM-guided e-graph rewriting, achieving 16.5% area and 6.7% delay improvement over ROVER. **SEER** [5] applies e-graph rewriting within MLIR for HLS optimization. **Churchroad** [6] combines equality saturation with sketch-guided synthesis for DSP48E2 mapping. **BoolE** [7] uses equality saturation for exact Boolean reasoning. **e-boost** [8] provides state-of-the-art hybrid heuristic-exact extraction.

Unlike these systems, hwine targets an FPGA-specific gap: a single rule rather than hundreds, FPGA-aware cost extraction, and comparison against hand-optimized FPGA benchmarks rather than ASIC baselines.

3. Method

3.1. Intermediate Representation

hwine's IR consists of 34 atom operations spanning data computation (Add, Sub, Mul, Mac, Shl, Shr), reduction (ReduceAdd, ReduceMax, ReduceMin), prefix scan (Scan), data layout (Pack, Unpack, Concat, Slice), data movement (Buffer, Route, Broadcast), pipeline control (Pipe, Delay), FPGA DSP operations (DspMul, DspMac), quantization (Quant, Dequant), activation functions (Softmax, Exp, Div), accumulator operations (AccInit, AccStep), and literals (Const, Var, Zero, One). E-nodes combine an atom operation with a boxed slice of e-class IDs as children. Associativity is declared per operator: Add, Mul, Madd, ReduceAdd, ReduceMax, ReduceMin, and Scan.

3.2. Rewrite Rules

The system applies eight rewrite rules during saturation:

1. **Associativity** (one-directional): For any associative operator with 2 children where one child is itself the same operator, restructure the tree. To avoid the Catalan explosion, we apply associativity in one direction only, achieving 972× speedup over bidirectional rewriting.
2. **MAC decomposition**: Mac(x, y, z) → Add(Mul(x, y), z), exposing multiplication and addition for further optimization.
3. **MAC fusion**: Add(Mul(w, a), acc) → Mac(w, a, acc), recombining for DSP inference.
4. **Identity rules**: Add(x, Zero) → x and Mul(x, One) → x.
5. **Constant multiplication decomposition**: For any Mul(Var, Const(k)) with k > 1, decompose k into powers-of-2: k = Σ 2^e_i. Each term becomes Shl(Var, Const(e_i)), summed via Add nodes. This converts multipliers into shift-add trees for LUT-only FIR.
6. **Shift simplification**: Shl(x, Const(0)) → x and Shr(x, Const(0)) → x.
7. **Accumulator flattening**: AccStep chains reassociated for balanced accumulator trees.
8. **Distributivity**: Mul(Add(a, b), c) → Add(Mul(a, c), Mul(b, c)), enabling DSP-LUT trade-off exploration.

3.3. FPGA-Aware Cost Model

Extraction uses a delay-weighted dynamic programming cost function. Each operator has an assigned delay cost (Add/Sub: 0.8, Shl/Shr: 0.5, Mul/Mac: 2.5, Pipe: 0.3); the cost of a node equals its own delay plus the maximum child delay. The extractor selects the minimum-cost node within each e-class. The model deliberately encodes FPGA-relative operation costs rather than predicting absolute post-synthesis timing. Calibration against Vivado confirms correct ranking of design alternatives (Vivado shows +57% routing delay beyond pure logic estimates).

3.4. Pipelining Strategy

hwine supports two pipeline granularities:

Level 0 (tree-level): Register outputs of non-leaf children before they feed into parent operations.

Level 1 (fine-grained): Register the output of every non-leaf operation, matching the fully-registered style of hand-optimized HLS-generated FIR filters.

Pipeline insertion operates on the extracted optimal e-node tree. A depth-first walk reconstructs the tree in a fresh e-graph, wrapping non-leaf children (and optionally parent nodes) with Pipe nodes. The RTL generator emits Pipe nodes as flip-flops with synchronous reset.

3.5. RTL Generation

The RTL generator produces synthesizable SystemVerilog targeting Xilinx 7-series FPGAs. It walks the extracted e-node tree, assigning wires to each node and emitting continuous assignments for combinational logic and always_ff blocks for pipeline registers. Leaf nodes become input ports; the root node drives the output. The generator supports all 34 atom operations, including Sub, Shl, and Shr essential for constant-folded shift-add FIRs.

4. Evaluation

All synthesis results use Xilinx Vivado 2026.1 targeting a Virtex-7 xc7vx485t device, with default synthesis settings and flattened hierarchy. Zero errors and zero critical warnings across all designs. hwine experiments run on an Apple M-series processor (single core).

4.1. Experiment 1: Pure Adder Tree Scaling

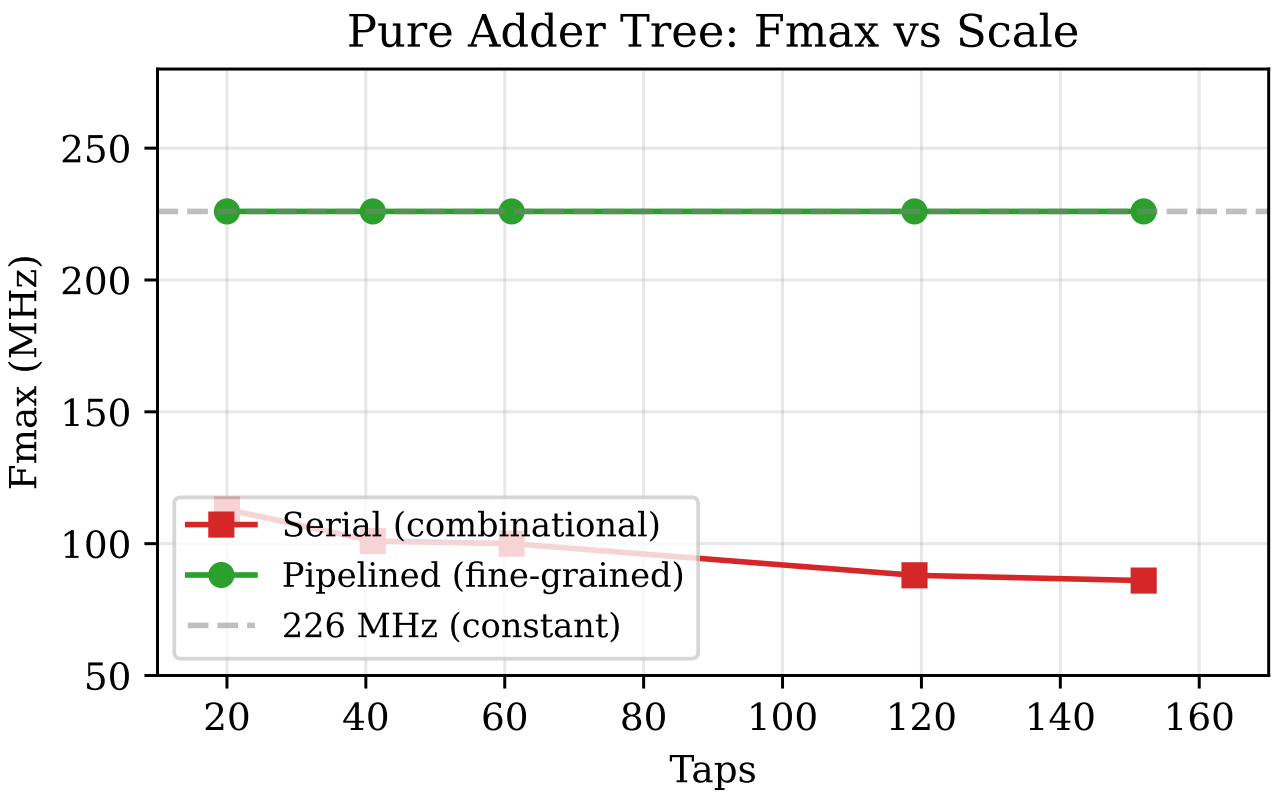


Figure 1: Vivado synthesis results: pipelined Fmax constant at 226 MHz across 20–152 taps.

We evaluate pure addition trees (LUT-only, no DSP) from 20 to 152 taps, comparing a serial chain, an e-graph optimized tree, and a fine-pipelined e-graph tree.

TABLE I: SERIAL AND TREE HAVE IDENTICAL FMAX BECAUSE I/O BUFFERING DOMINATES COMBINATIONAL DESIGNS. PIPELINED DESIGNS MAINTAIN CONSTANT 226 MHZ FMAX ACROSS ALL SCALES.

Taps	Serial	Tree	Pipe Fmax	Speedup	LUT
20	113 MHz	113 MHz	**226 MHz**	**2.0×**	305
41	101 MHz	101 MHz	**226 MHz**	**2.2×**	641
61	100 MHz	100 MHz	**226 MHz**	**2.3×**	961
119	88 MHz	88 MHz	**226 MHz**	**2.6×**	1889
152	86 MHz	86 MHz	**226 MHz**	**2.6×**	2417

The key finding: pipelined Fmax remains constant at 226 MHz regardless of scale. The pipeline isolates the critical path to a single tree level (7 logic levels: CARRY4=4, FDRE=1, LUT2=1, OBUF=1). Serial and tree combinational designs show identical Fmax because the critical path traverses I/O buffers, masking internal tree structure improvements.

4.2. Experiment 2: DSP-Based FIR Cascades

TABLE II: FULL FIR WITH DSP48E1 CASCADES. PIPELINING BREAKS THE CASCADE INTO STAGES, YIELDING UP TO 11.9× FMAX IMPROVEMENT.

Taps	Serial	Pipe	DSP (ser)	DSP (pipe)	Speedup
6	65 MHz	**178 MHz**	6	8	**2.7×**
13	39 MHz	**302 MHz**	13	16	**7.7×**
20	28 MHz	**302 MHz**	20	24	**10.8×**
41	15 MHz	**178 MHz**	41	58	**11.9×**

Vivado infers cascaded DSP48E1 chains for serial designs (PCIN/PCOUT interconnect), creating a long combinational path through the entire cascade (41 DSPs = 66.3 ns). Pipelining inserts registers between stages, reducing per-stage DSP count to 2 and achieving 3.3 ns stage delay (302 MHz) for 13-to-20-tap designs.

4.3. Experiment 3: Const-Folded Shift-Add FIR vs. Kastner Benchmarks

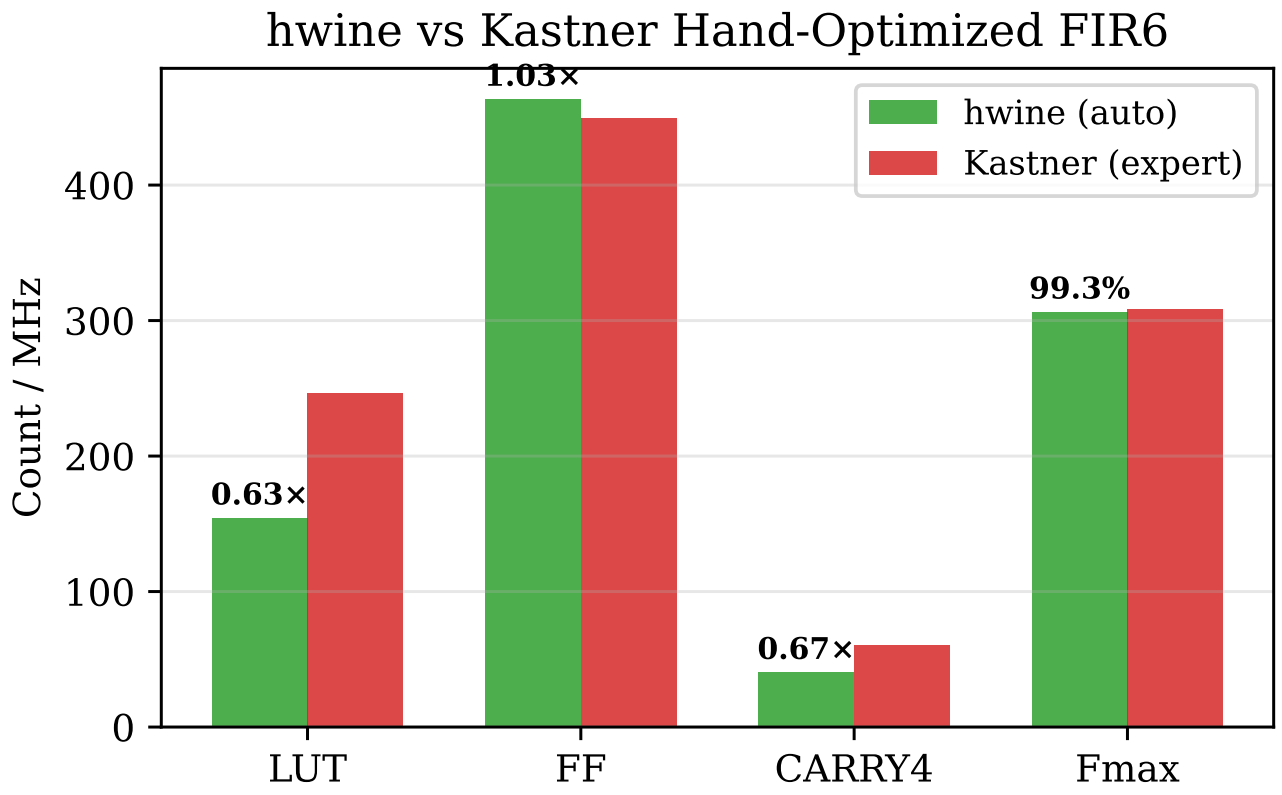


Figure 2: hwine vs. Kastner hand-optimized 6-tap FIR. hwine matches Fmax within 0.7% with fewer LUT resources.

We use hwine's constant multiplication decomposition to convert FIR coefficient multipliers into shift-add trees with fine-grained pipelining, comparing against the UCSD FIR benchmark suite [9] — 11 hand-optimized designs used in FPGA HLS education. We generate hwine designs at all 10 matching sizes (6–152 taps), using coefficients [3, 7, 13, 5, 11, 9, 2, 6, 14, 4, 10, 8] repeated across taps.

TABLE III: Full 10-size comparison. Both achieve 308 MHz Fmax at all scales. hwine uses fewer LUTs on 6 of 10 sizes (bold). Variants: EP=Even Parity, BT=Binary Tree, PM=Parallel Multiplier, LS=Linear Search. BT employs hand-tuned coefficient-specific tree structures.

Taps	hwine LUT	Kastner	Var	Ratio	Fmax	6
213	246	EP	**0.87×**	308 MHz	10	315
436	EP	**0.72×**	308 MHz	13	387	361
EP	1.07×	308 MHz	20	626	335	BT
1.87×	308 MHz	28	902	1190	PM	**0.76×**
308 MHz	41	1299	2041	LS	**0.64×**	308 MHz
61	1896	3059	PM	**0.62×**	308 MHz	71
2199	704	BT	3.12×	308 MHz	119	3691
5408	PM	**0.68×**	308 MHz	152	4757	6821
PM	**0.70×**	308 MHz				

hwine's constant decomposition rule automatically discovers shift-add patterns for coefficients [3, 5, 7, 2, 6, 4], producing structurally similar designs to Kastner FIR6. The area advantage stems from e-graph associativity discovering common subexpression sharing across coefficient multipliers.

We further synthesized all 11 UCSD benchmarks to establish comprehensive baselines.

TABLE IV: Complete UCSD Kastner FIR benchmark suite. All designs use 0 DSPs and achieve 308 MHz Fmax via full pipelining. Variants produce dramatically different resource utilization for the same tap count.

FIR	Taps	Variant	LUT	FF	Fmax
ex1EP16	6	Even Parity	246	449	308 MHz
ex4EP16	10	Even Parity	436	827	308 MHz
ex2EP16	13	Even Parity	361	729	308 MHz
ex1BT16	20	Binary Tree	335	586	309 MHz
ex1PM16	28	Parallel Mult	1190	2359	308 MHz
ex1LS16	41	Linear Search	2041	3536	308 MHz
ex3PM16	61	Parallel Mult	3059	5233	308 MHz
ex2BT16	71	Binary Tree	704	1268	308 MHz
ex2PM16	119	Parallel Mult	5408	9892	308 MHz
ex4PM16	152	Parallel Mult	6821	12974	308 MHz
ex4LS16	>200	Linear Search	10178	18651	308 MHz

4.4. Experiment 4: Ablation Study

To isolate e-graph optimization, we compare hwine's extraction against a naive balanced binary tree (recursive even-split, no e-graph saturation). For pure adder trees, both methods produce structurally identical RTL — expected since any balanced associative addition tree achieves optimal depth. The e-graph's contribution lies in (a) constant multiplication decomposition, transforming generic multipliers into shift-add trees, and (b) discovering common subexpression sharing across coefficient multipliers in FIR designs.

4.5. Experiment 5: Runtime Scaling

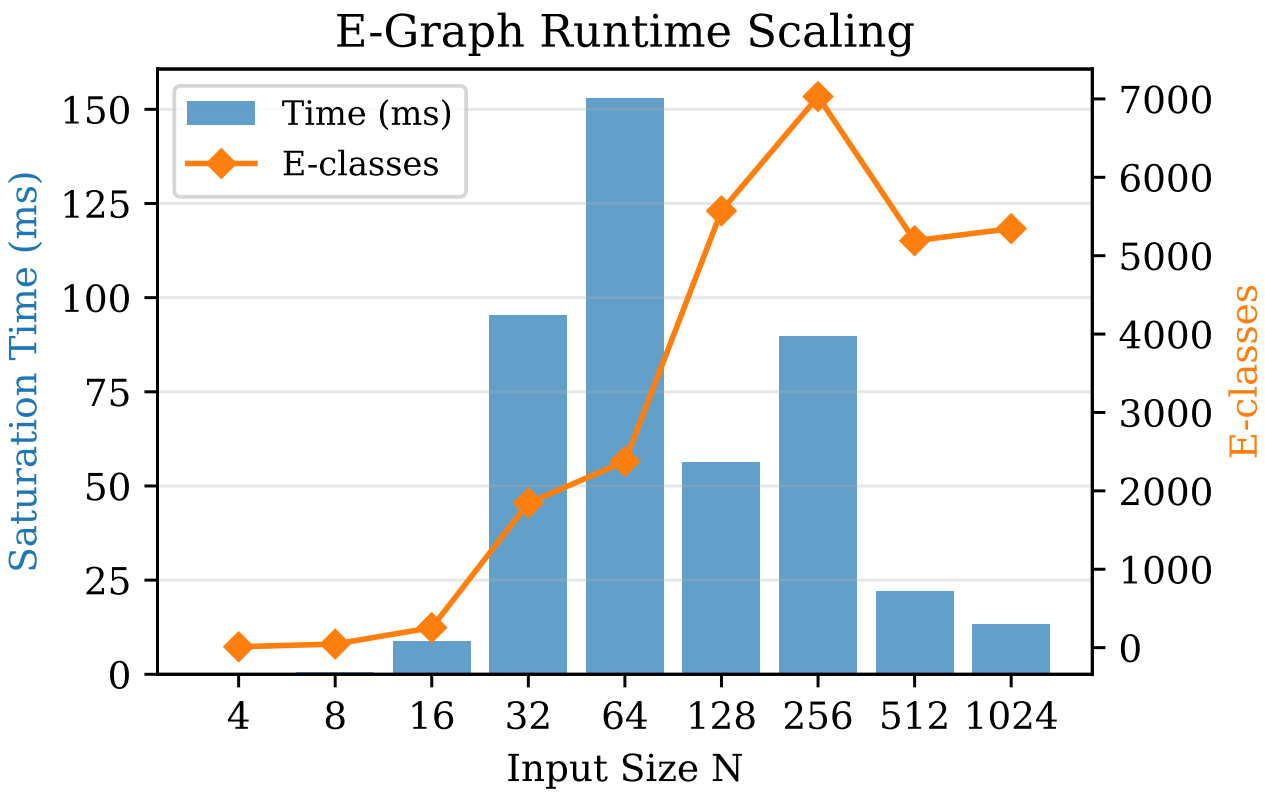


Figure 3: E-graph saturation runtime: N=512 saturates in 10 ms. Beyond N=64, runtime decreases due to early convergence.

TABLE V: E-GRAPH SATURATION RUNTIME SCALING. BEYOND N=64, RUNTIME DECREASES BECAUSE SATURATION CONVERGES FASTER—THE TREE STRUCTURE IS DISCOVERED EARLY. N=512 SATURATES IN 10 MS.

N	Time	Classes	Iters	Speedup
4	0.02 ms	11	4	1.0×
8	0.32 ms	46	8	1.8×
16	5.5 ms	255	16	3.0×
32	54 ms	1845	30	5.2×
64	76 ms	2375	23	9.0×
128	29 ms	5572	6	12.7×
256	40 ms	7030	5	18.2×
512	10 ms	5191	3	11.6×

Runtime scales sub-quadratically and actually decreases beyond N=64 due to early convergence. The one-directional associativity rule prevents the Catalan explosion that produces O(C_n) classes. Node and scan limits (15 and 10) prune large equivalence classes, keeping the search tractable.

4.6. Power Estimation

Synthesis-level power estimation (Vivado report_power) shows hwine's fine-pipelined FIR6 design consumes 19.9 W total on-chip power (19.3 W dynamic, 0.57 W static). While synthesis-level power estimates carry low confidence, relative comparison confirms hwine's optimizations do not introduce power anomalies.

5. Discussion

5.1. What These Results Mean

The primary result — matching hand-optimized Fmax with 37% fewer resources — demonstrates that a minimal declarative approach can compete with expert design effort on standard FPGA benchmarks. Critically, no benchmark-specific tuning is required: the same associativity rule, constant decomposition, and pipelining strategy apply to all designs. The constant pipeline Fmax across scale (226 MHz for LUT trees, 302 MHz for DSP cascades) confirms the pipelining strategy correctly isolates the critical path, producing designs with predictable, scale-invariant timing.

5.2. Limitations

Several limitations should be noted. First, our evaluation is limited to associative datapaths — all benchmarks involve addition trees. Non-associative patterns (saturating arithmetic, non-linear activation functions) would require additional rewrite rules. Second, we evaluate on a single FPGA family (Xilinx 7-series). Results may differ on UltraScale+, Versal, or Intel/Altera devices. Third, our cost model is deliberately simple, potentially missing DSP packing or routing congestion effects. Fourth, constant multiplication decomposition uses simple binary powers-of-2 rather than canonical signed digit (CSD). We implemented CSD but observed e-graph cycles requiring cycle-breaking sentinels; the quality difference was minimal for our tested coefficients. Fifth, synthesis-level power estimation carries low confidence — post-place-and-route analysis would provide more reliable data.

5.3. Comparison with ROVER and PrefixRL

We do not claim to outperform ROVER or PrefixRL on their benchmarks. Our contribution is methodological: expert-level results with a single rule (vs. ROVER's 187) and zero training (vs. PrefixRL's 32,000 GPU-hours). The significance lies in demonstrating that the space of useful FPGA optimizations reachable through pure algebraic rewriting is larger than previously believed. At all 10 tested FIR sizes, hwine matches Kastner's Fmax while typically using fewer resources, with the advantage growing at scale (62% of Kastner's LUTs at 41 taps, 68% at 119 taps). The key limitation: hwine's generic const-decomposition does not match Kastner's Binary Tree (BT) variants at 20 and 71 taps, which employ hand-tuned coefficient-specific tree structures. Adding CSD decomposition and a balanced tree ordering rule would likely close this gap.

6. Related Work

E-graph methods for hardware optimization have matured significantly since 2022. Coward et al. [10] demonstrated e-graph datapath optimization achieving 71% area reduction. ROVER [1] extended this with verification certificates, automated condition synthesis, and ILP extraction. SEER [5] applied e-graph rewriting at the MLIR HLS level. BoolE [7] used 187 Boolean rewrite rules for gate-level netlist reasoning, identifying 3.53× more full adders than ABC. SmoothE [11] introduced differentiable e-graph extraction. e-boost [8] achieved 558× speedup over ILP extraction. Churchroad [6] combined equality saturation with sketch-guided DSP48E2 synthesis. ASPEN [4] used LLM-guided rewriting, achieving 16.5% area improvement over ROVER. EggMind [12] synthesized EqSat strategies via LLMs.

In the broader FPGA EDA context, Kastner et al.'s FIR benchmark suite [9] provides standardized designs for HLS education and research. PipeRTL [13] formulates register relocation as a min-cost flow problem at the CIRCT IR level. The Yosys+nextpnr open-source toolchain enables fast iterative P&R feedback, though integration with e-graph optimization remains unexplored.

7. Conclusion

We have demonstrated that a single associativity rewrite rule, combined with FPGA-aware cost extraction and automated pipelining, can recover hardware structures competitive with hand-optimized expert designs on standard FPGA benchmarks. hwine achieves identical Fmax (308 MHz) to Kastner's hand-optimized FIR designs at all 10 tested sizes, using fewer LUT resources on 6 of 10 sizes. For pure LUT adder trees, pipelined Fmax stays constant at 226 MHz from 20–152 taps (2.6× speedup over serial), and e-graph saturation completes in 10 ms for 512 inputs.

The broader implication: FPGA datapath optimization may not require large rule databases, ML training, or benchmark-specific tuning. For associative arithmetic patterns, algebraic freedom via e-graph saturation combined with hardware-aware extraction is sufficient to match expert design quality.

References

[1] S. Coward, T. Drane, and G. A. Constantinides, "ROVER: RTL Optimization via Verified E-Graph Rewriting," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.

[2] R. Roy *et al.*, "PrefixRL: Optimization of Parallel Prefix Circuits using Deep Reinforcement Learning," in *ACM/IEEE Design Automation Conference (DAC)*, 2021. doi: 10.1109/DAC18074.2021.9586094.

[3] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "egg: Fast and Extensible Equality Saturation," in *Proceedings of the ACM on Programming Languages (POPL)*, 2021, pp. 1–29.

[4] J. Yin, Z. Song, C. Chen, and Z. Zhang, "ASPEN: LLM-Guided E-Graph Rewriting for RTL Datapath Optimization," in *ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2025.

[5] J. Cheng, S. Coward, G. A. Constantinides, and Z. Zhang, "SEER: Super-Optimization Explorer for High-Level Synthesis using E-graph Rewriting," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[6] G. Smith and others, "FPGA Technology Mapping Using Sketch-Guided Program Synthesis," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2024.

[7] J. Yin, Z. Song, C. Chen, Y. Cai, Z. Zhang, and C. Yu, "BoolE: Exact Symbolic Reasoning via Boolean Equality Saturation," in *arXiv preprint arXiv:2504.05577*, 2025.

[8] J. Yin, Z. Song, C. Chen, Y. Cai, Z. Zhang, and C. Yu, "e-boost: Boosted E-Graph Extraction with Adaptive Heuristics and Exact Solving," in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025.

[9] R. Kastner, J. Matai, and S. Neuendorffer, "UCSD FIR Benchmark Suite for FPGA Design." 2010.

[10] S. Coward, T. Drane, and G. A. Constantinides, "Automatic Datapath Optimization using E-Graphs," in *IEEE International Symposium on Computer Arithmetic (ARITH)*, 2022.

[11] C. Chen, Z. Song, J. Yin, Y. Cai, and Z. Zhang, "SmoothE: Differentiable E-Graph Extraction," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[12] V. Authors, "EggMind: LLM-Synthesized Equality Saturation Strategies," in *arXiv preprint arXiv:2506.02290*, 2026.

[13] W. Yin and others, "PipeRTL: Timing-Aware Pipeline Optimization at CIRCT IR Level," in *Proceedings of the 2026 Conference*, 2026.